CS535: Algorithm Design and Analysis

Basic Data Structures

Prof. Rohit P. Singh

Department of Computer Science

University of Wisconsin, Milwaukee

September 16, 2024

Readings: Chapter 2.1, 2.2

Outline

- Stack
- Queue
- Vector
- List
- Sequence

Abstract Data Types (ADTs)

- An abstract data type (ADT) is an abstraction of a data structure
- An ADT specifies:
  - Data stored
  - Operations on the data
  - Error conditions associated with operations (**exceptions**)
- *Example*: ADT modeling a simple stock-trading system
  - The data stored are buy/sell orders
  - The operations supported are
    - order buy (stock, shares, price)
    - order sell (stock, shares, price)
    - void cancel (order)
  - Error conditions (**exceptions**):
    - Buy/sell a nonexistent stock
    - Cancel a nonexistent order

The Stack ADT

- The Stack ADT stores arbitrary objects
- Insertions and deletions follow the last-in first-out scheme
- Main stack operations:
  - **push**(object): inserts an element
  - object **pop**(): removes and returns the last inserted element
- Other stack operations:
  - object **top**(): returns the last inserted element without removing it
  - integer **size**(): returns the number of elements stored
  - boolean **isEmpty**(): indicates whether no elements are stored

Applications:

Page-visited history in a Web browser
Undo sequence in a text editor
Chain of method calls in the Java Virtual Machine

Array-based Stack

- A simple way of implementing the Stack ADT uses an array
- We add elements from left to right
- A variable keeps track of the index of the top element

Algorithm *size*()
  return $t + 1$

Algorithm *pop*()
  if *isEmpty*() **then**
    Indicate a stack-empty error
  else
    $t \leftarrow t - 1$
    return $S[t + 1]$

Array-based Stack (contd.)

- The array storing the stack elements may become full
- A push operation will then throw a FullStack exception

Algorithm *push*(o)
  if $t = S.length - 1$ **then**
    Indicate a stack-full error
  else
    $t \leftarrow t + 1$
    $S[t] \leftarrow o$

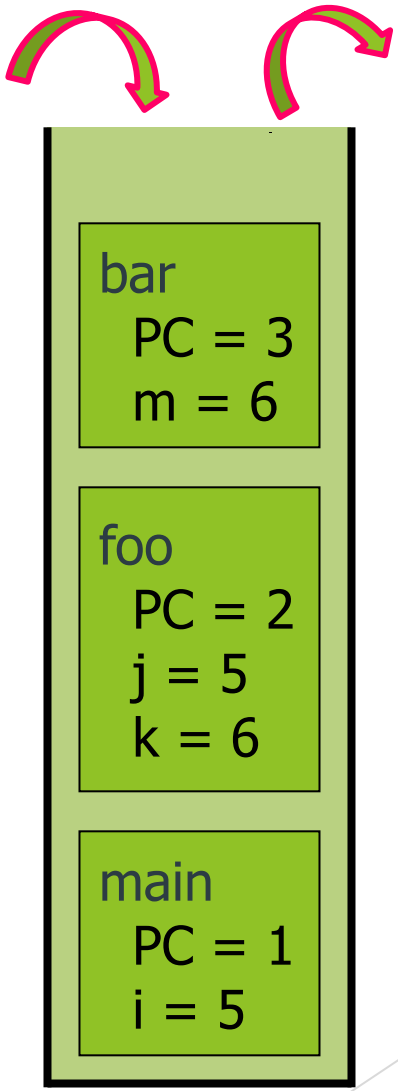
Application: Function Calls

- The Java Virtual Machine (JVM) keeps track of the chain of active methods with a stack
- When a method is called, the JVM pushes on the stack a frame containing
  - Local variables and return value
  - Program counter (PC), keeping track of the statement being executed
- When a method ends, its frame is popped from the stack and control is passed to the method on top of the stack

```
main() {
   int i = 5;
   foo(i);
   }

foo(int j) {
   int k;
   k = j+1;
   bar(k);
   }

bar(int m) {
   …
   }
```



Performance and Limitations

- Performance
  - Let n be the number of elements in the stack
  - The space used is $O(n)$
  - Each operation runs in time $O(1)$
- Limitations
  - The maximum size of the stack must be defined ahead of time and cannot be changed
  - Trying to push a new element into a full stack causes an implementation-specific exception

Growable Array-based Stack

- In a push operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one

- How large should the new array be?
  - incremental strategy: increase the size by a constant c
  - doubling strategy: double the size

```
Algorithm push(o)
  if t = S.length – 1 then
    A ← new array of
          size …
    for i ← 0 to t do
      A[i] ← S[i]
      S ← A
  t ← t + 1
  S[t] ← o
```

Comparison of the Strategies

- We compare the incremental strategy and the doubling strategy by analyzing the total time $T(n)$ needed to perform a series of n push operations

- We assume that we start with an empty stack represented by an array of size 1

- We call *amortized time* of a push operation the average time taken by a push over the series of operations, i.e., $T(n)/n$

Incremental Strategy Analysis

- We replace the array $k = n/c$ times
- The total time $T(n)$ of a series of n push operations is proportional to

Push

Making new arrays

$$n + [c + 2c + 3c + \ldots + kc] =$$

$$n + c(1 + 2 + 3 + \ldots + k) =$$

$$n + ck(k + 1)/2$$

- Since c is a constant, $T(n)$ is $O(n + k^2)$, i.e., $O(n^2)$
- The amortized time of a push operation is $O(n)$

Doubling Strategy Analysis

- We replace the array $k = \log_2 n$ times

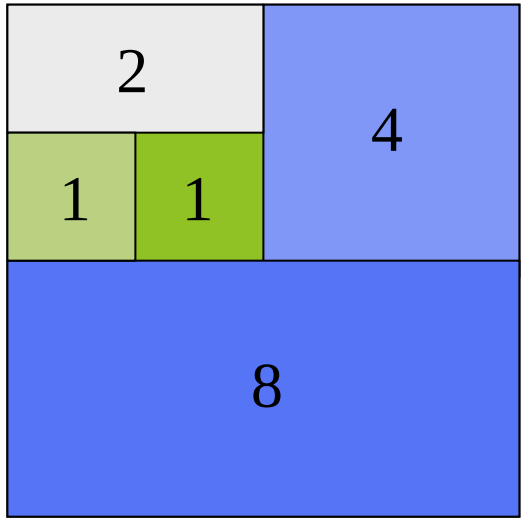
- The total time $T(n)$ of a series of n push operations is proportional to

$$n + 1 + 2 + 4 + 8 + \ldots + 2^k =$$

$$n + 2^{k+1} - 1 = 3n - 1$$

- $T(n)$ is $O(n)$
- The amortized time of a push operation is $O(1)$

geometric series



The Queue ADT

- The Queue ADT stores arbitrary objects
- Insertions and deletions follow the first-in first-out scheme
- Insertions are at the rear of the queue and removals are at the front of the queue
- Main queue operations:
  - **enqueue**(object): inserts an element at the end of the queue
  - object **dequeue**(): removes and returns the element at the front of the queue
- Additional operations:
  - object **front**(): returns the element at the front without removing it
  - integer **size**(): returns the number of elements stored
  - boolean **isEmpty**(): indicates whether no elements are stored
- Exceptions
  - Attempting the execution of **dequeue** or **front** on an empty queue throws an EmptyQueueException

Applications:

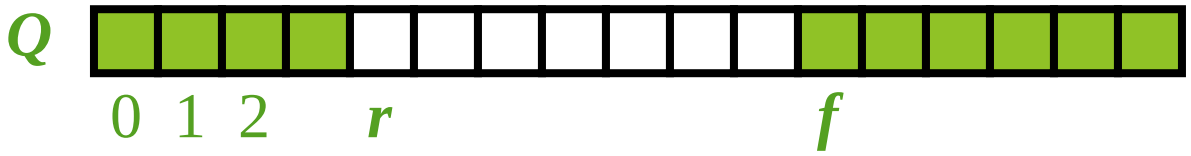
Waiting lists
Access to shared resources (e.g., printer)
Multiprogramming

Array-based Queue

- Use an array of size N in a circular fashion
- Two variables keep track of the front and rear
  - f index of the front element
  - r index immediately past the rear element
- Array location r is kept empty

normal configuration

wrapped-around configuration

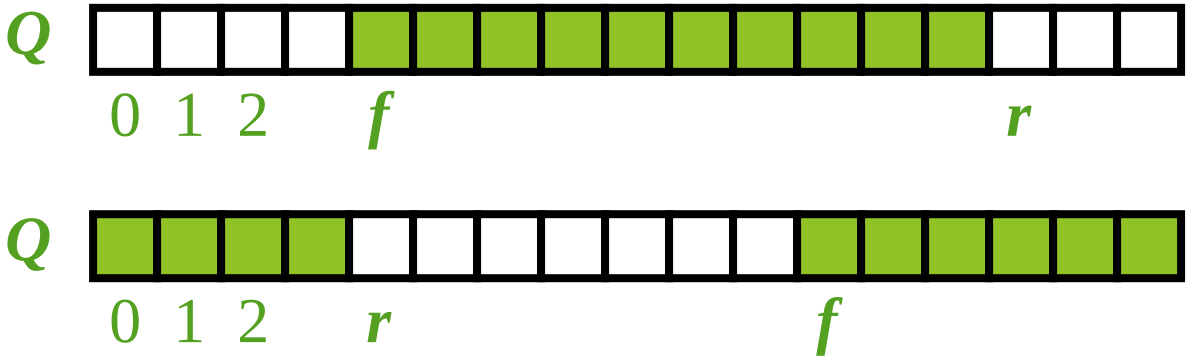


Queue Operations

- We use the modulo operator (remainder of division)

Algorithm ***size***()
  return $(r - f) \bmod N$

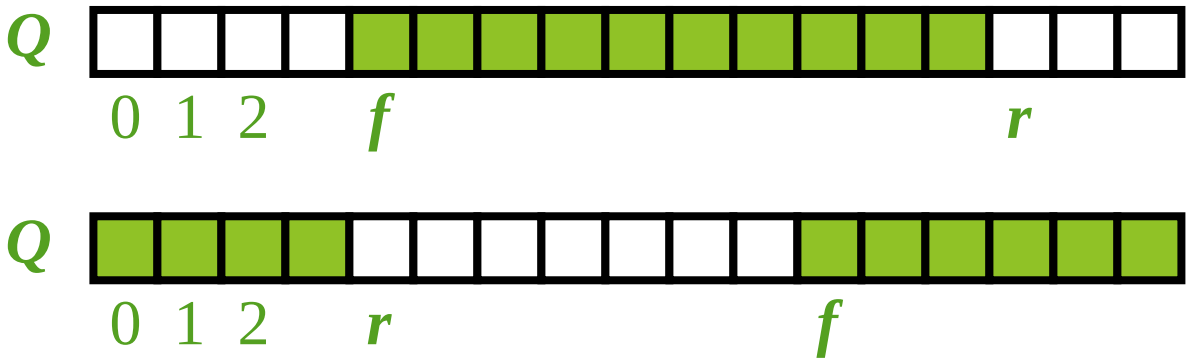
Algorithm ***isEmpty***()
  return $(f = r)$



Queue Operations (Contd.)

- Operation **enqueue** throws an exception if the array is full
- This exception is implementation-dependent

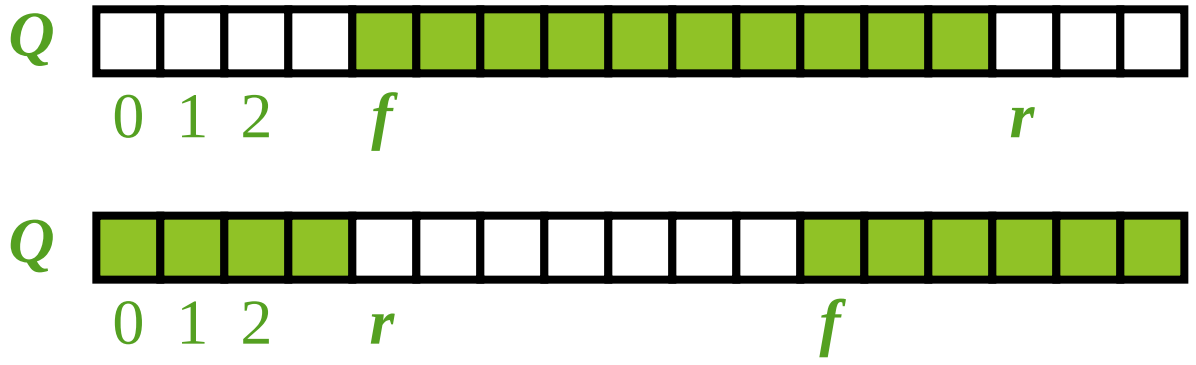
Algorithm *enqueue*(o)
  if $size() = N - 1$ **then**
    throw *FullQueueException*
  else
    $Q[r] \leftarrow o$
    $r \leftarrow (r + 1) \bmod N$



Queue Operations (Contd.)

- Operation **dequeue** throws an exception if the queue is empty
- This exception is specified in the queue ADT

```
Algorithm dequeue()
  if isEmpty() then
    throw EmptyQueueException
  else
    o ← Q[f]
    f ← (f + 1) mod N
  return o
```



Growable Array-based Queue

- In an **enqueue** operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one
- Similar to what we did for an array-based stack
- The **enqueue** operation has amortized running time
  - $O(n)$ with the incremental strategy
  - $O(1)$ with the doubling strategy

The Vector ADT

- The Vector ADT extends the notion of array by storing a sequence of arbitrary objects
- An element can be accessed, inserted or removed by specifying its *rank* (number of elements preceding it)
- An exception is thrown if an incorrect rank is specified (e.g., a negative rank)

- Main vector operations:
  - object **elemAtRank**(integer r): returns the element at rank r without removing it
  - object **replaceAtRank**(integer r, object o): replace the element at rank with o and return the old element
  - **insertAtRank**(integer r, object o): insert a new element o to have rank r
  - object **removeAtRank**(integer r): removes and returns the element at rank r
- Additional operations
  - **size**() and **isEmpty**()

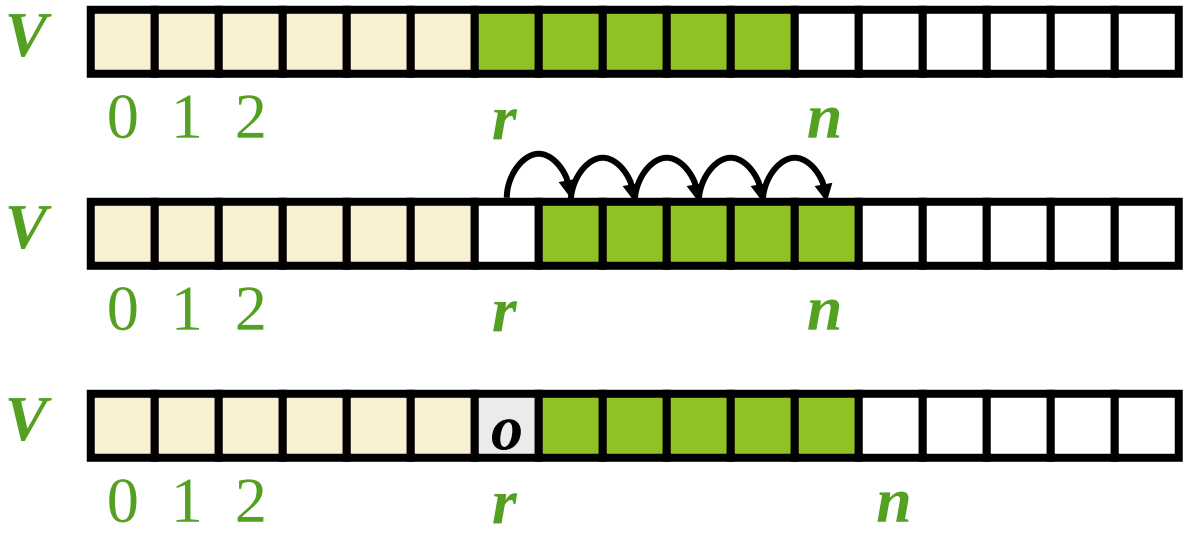
Array-based Vector

- Use an array V of size N
- A variable n keeps track of the size of the vector (number of elements stored)
- Operation $elemAtRank(r)$ is implemented in $O(1)$ time by returning $V[r]$

Insertion

- In operation $\boldsymbol{insertAtRank}(\boldsymbol{r}, \boldsymbol{o})$, we need to make room for the new element by shifting forward the $\boldsymbol{n} - \boldsymbol{r}$ elements $\boldsymbol{V}[\boldsymbol{r}], \ldots, \boldsymbol{V}[\boldsymbol{n} - 1]$

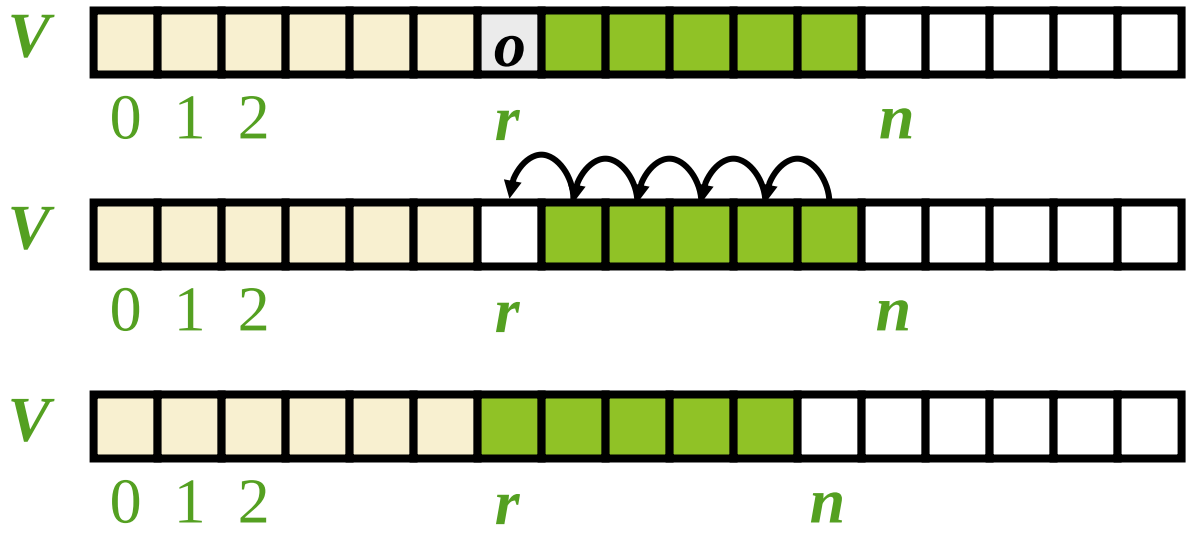
- In the worst case $(\boldsymbol{r} = 0)$, this takes $\boldsymbol{O}(\boldsymbol{n})$ time



Deletion

- In operation $\boldsymbol{removeAtRank(r)}$, we need to fill the hole left by the removed element by shifting backward the $\boldsymbol{n} - \boldsymbol{r} - 1$ elements $\boldsymbol{V}[\boldsymbol{r} + 1], \ldots, \boldsymbol{V}[\boldsymbol{n} - 1]$

- In the worst case $(\boldsymbol{r} = \boldsymbol{0})$, this takes $\boldsymbol{O}(\boldsymbol{n})$ time



Performance

- In the array based implementation of a Vector
  - The space used by the data structure is $O(n)$
  - $size$, $isEmpty$, $elemAtRank$ and $replaceAtRank$ run in $O(1)$ time
  - $insertAtRank$ and $removeAtRank$ run in $O(n)$ time

- If we use the array in a circular fashion, $insertAtRank(0)$ and $removeAtRank(0)$ run in $O(1)$ time

- In an $insertAtRank$ operation, when the array is full, instead of throwing an exception, we can replace the array with a larger one

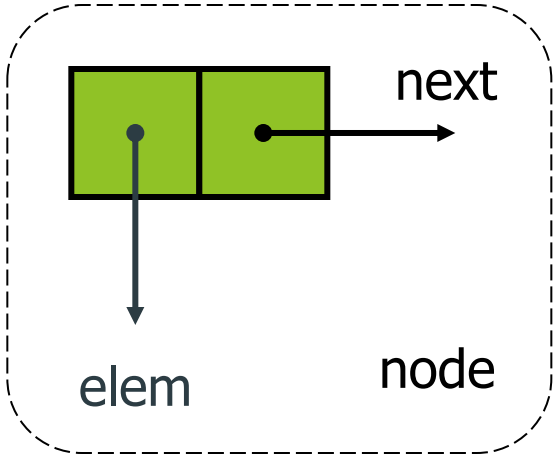
List ADT

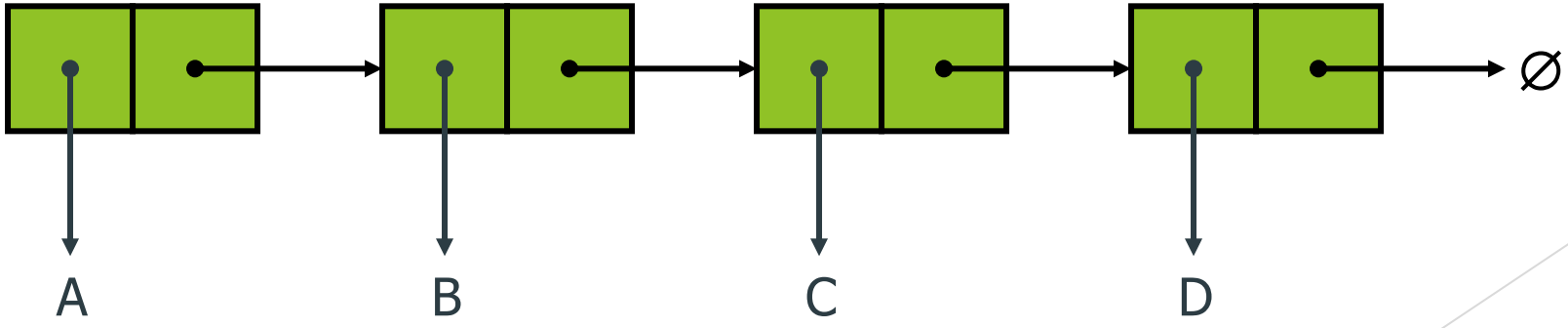
- The List ADT models a sequence of positions storing arbitrary objects
- It establishes a before/after relation between positions
- Generic methods:
  - **size**(), **isEmpty**()
- Query methods:
  - **isFirst**(p), **isLast**(p)
- Accessor methods:
  - **first**(), **last**()
  - **before**(p), **after**(p)
- Update methods:
  - **replaceElement**(p, o), **swapElements**(p, q)
  - **insertBefore**(p, o), **insertAfter**(p, o),
  - **insertFirst**(o), **insertLast**(o)
  - **remove**(p)

Singly Linked List

- A singly linked list is a concrete data structure consisting of a sequence of nodes

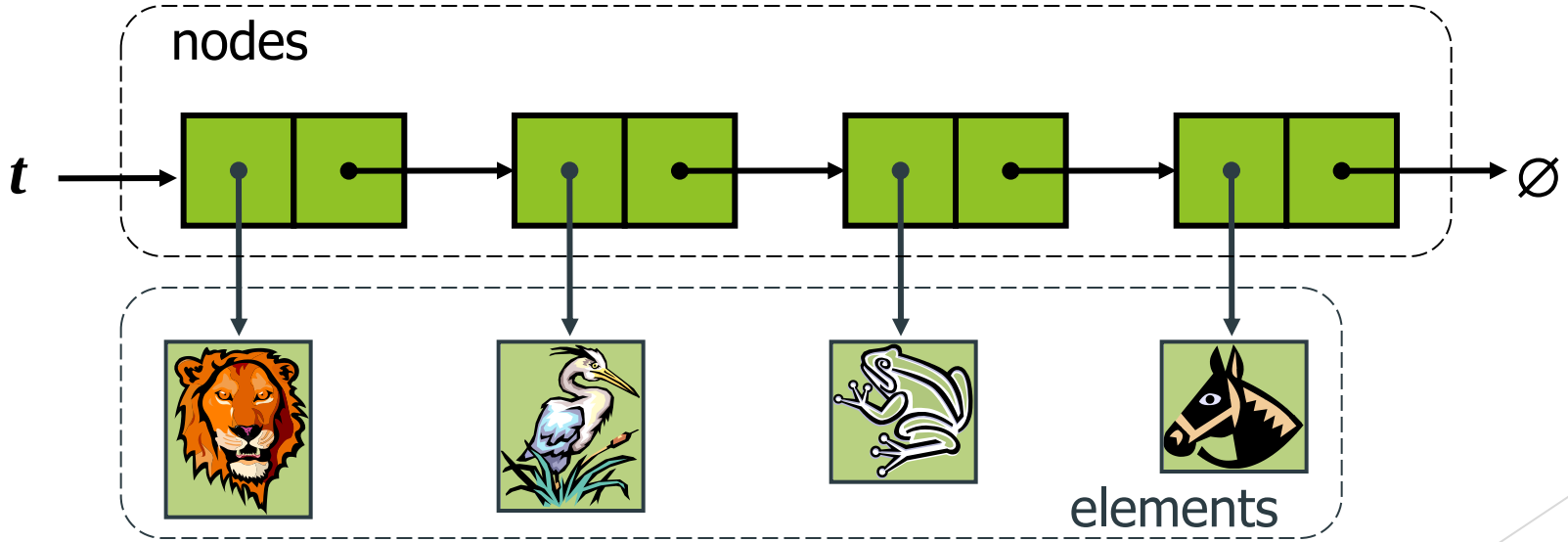
- Each node stores
  - element
  - link to the next node





Stack with a Singly Linked List

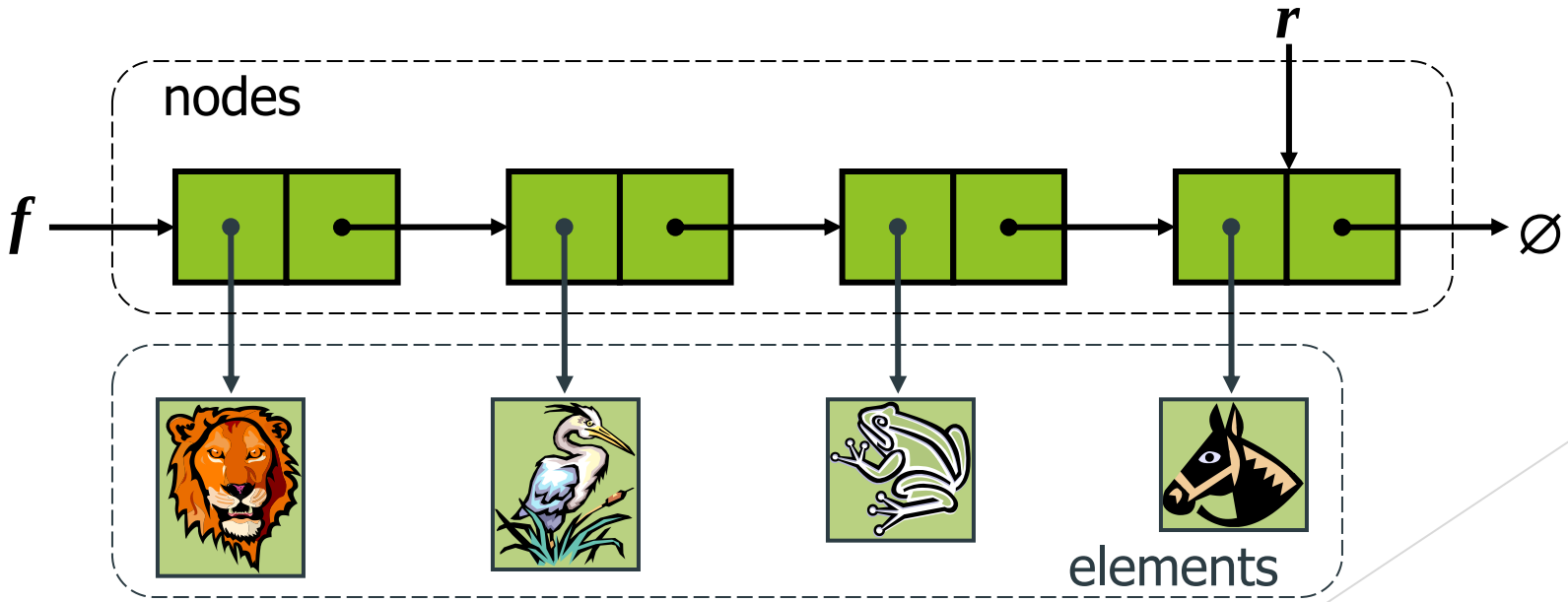
- We can implement a stack with a singly linked list
- The top element is stored at the first node of the list
- The space used is $O(n)$ and each operation of the Stack ADT takes $O(1)$ time



Queue with a Singly Linked List

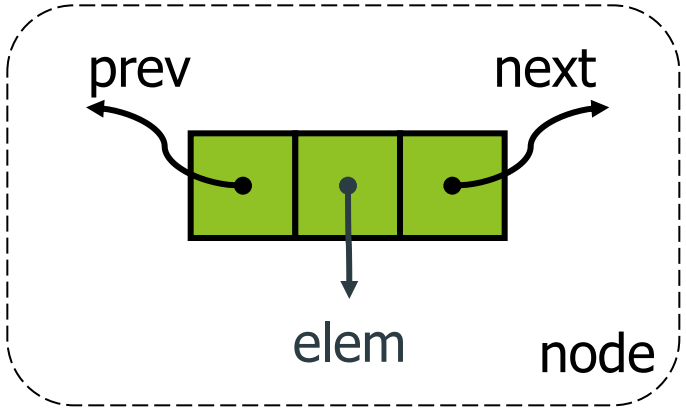
- We can implement a queue with a singly linked list
  - The front element is stored at the first node
  - The rear element is stored at the last node

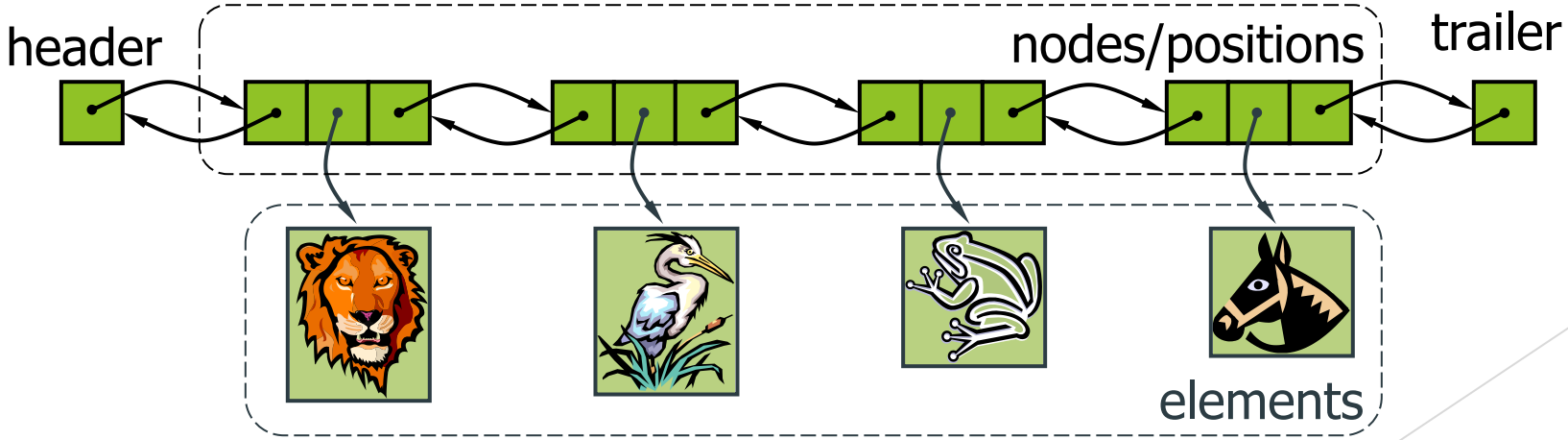
- The space used is $O(n)$ and each operation of the Queue ADT takes $O(1)$ time



Doubly Linked List

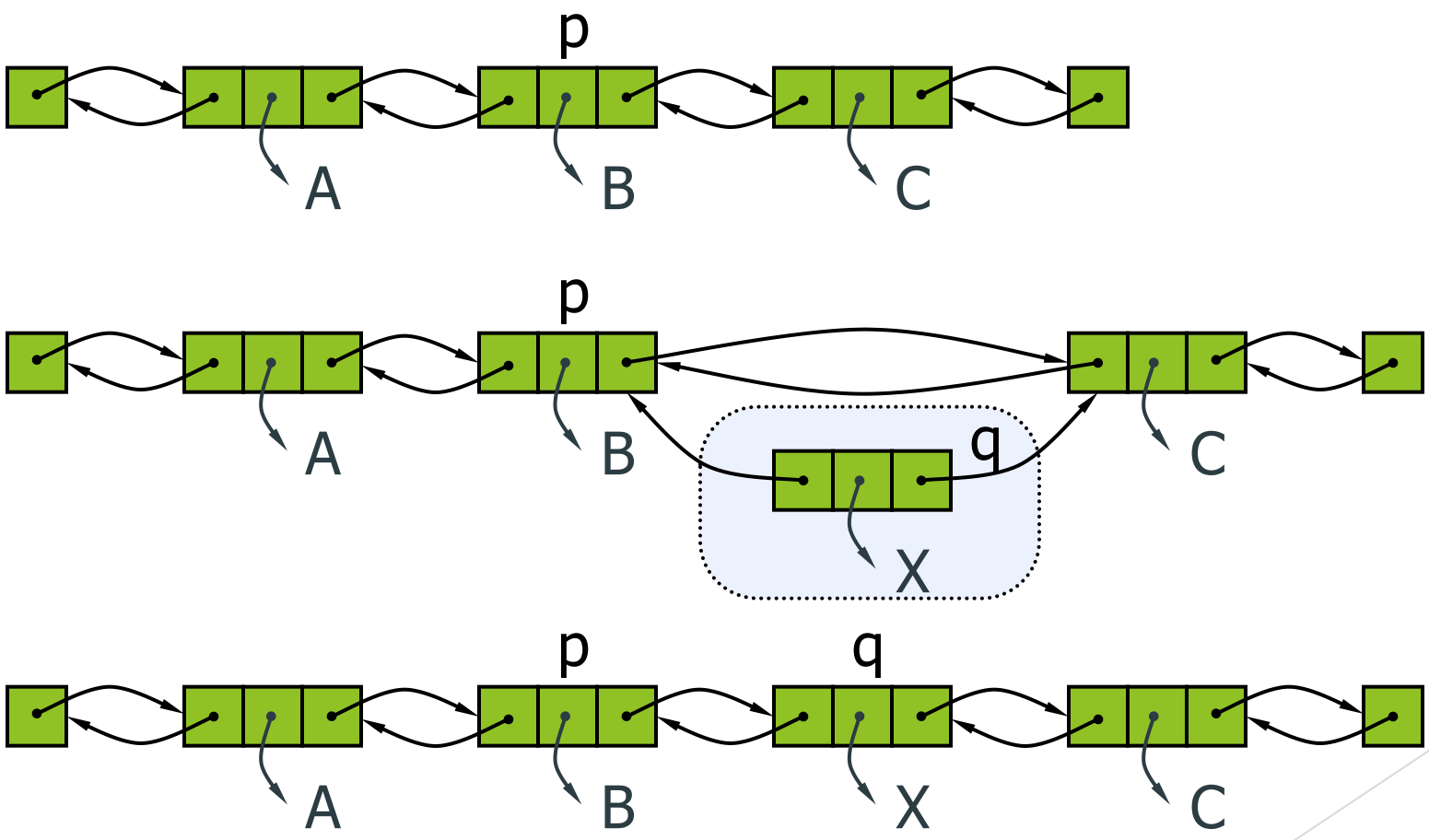
- A doubly linked list provides a natural implementation of the List ADT
- Nodes implement Position and store:
  - element
  - link to the previous node
  - link to the next node
- Special trailer and header nodes





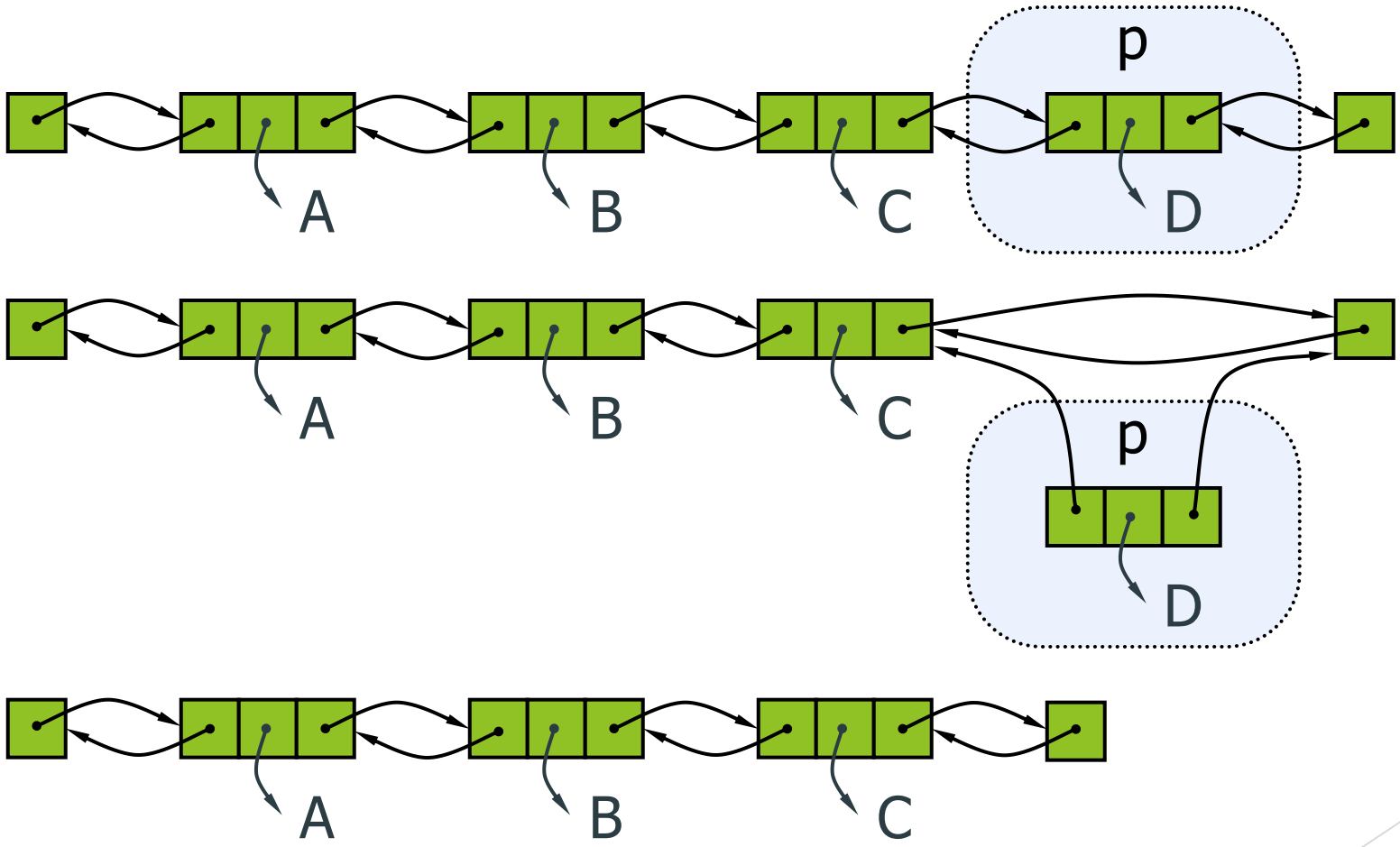
Insertion

- We visualize below the operation **insertAfter(p**, X), which returns position q



Deletion

- We visualize below the **remove**(p), where p = last()



Performance

- In the implementation of the List ADT by means of a doubly linked list
  - The space used by a list with n elements is $O(n)$
  - The space used by each position of the list is $O(1)$
  - **All the above-mentioned operations of the List ADT run in $O(1)$ time**

Sequence ADT

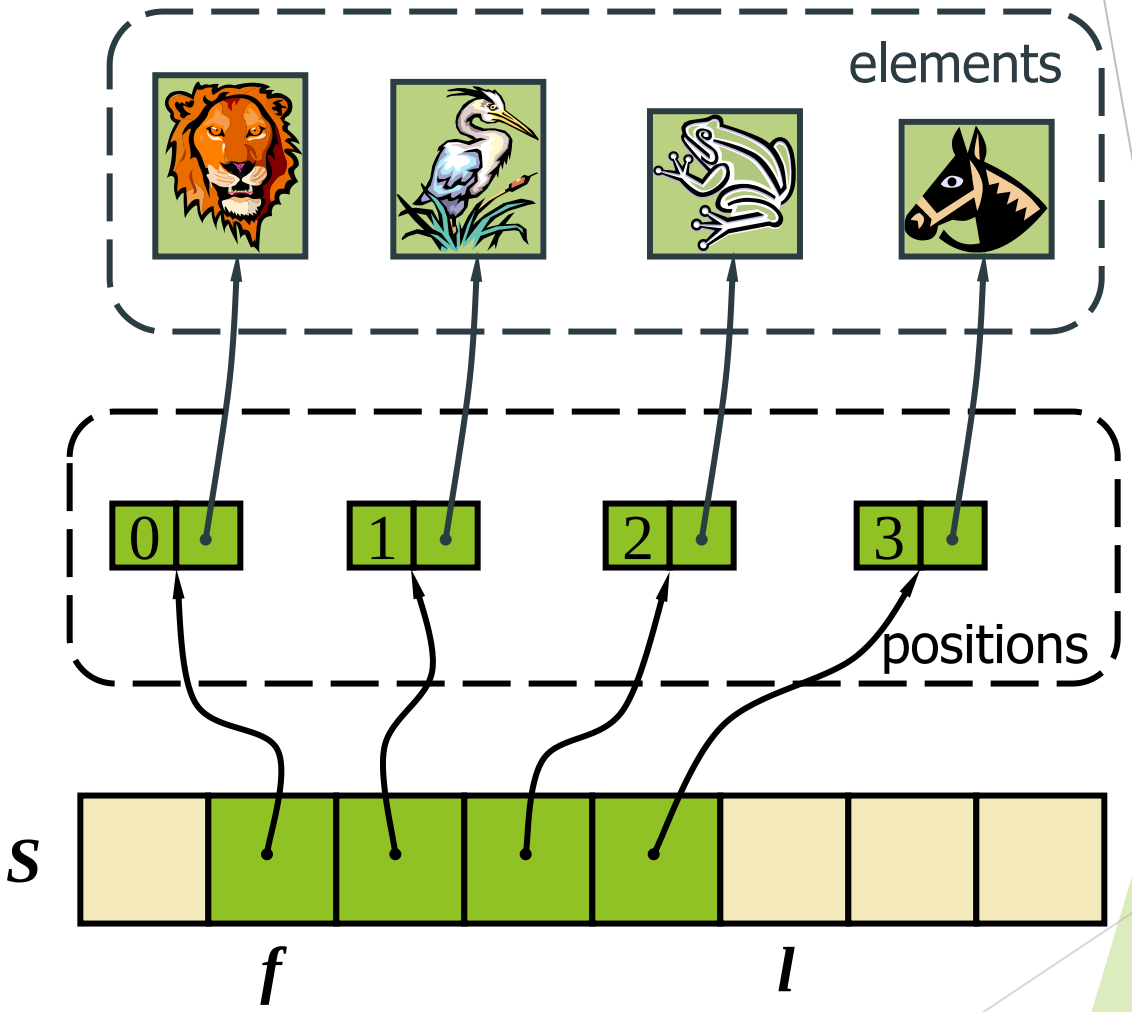
- The Sequence ADT is the union of the Vector and List ADTs
- Elements accessed by
  - Rank, or
  - Position
- Generic methods:
  - **size**(), **isEmpty**()
- Vector-based methods (rank):
  - **elemAtRank**(r)
  - **replaceAtRank**(r, o)
  - **insertAtRank**(r, o)
  - **removeAtRank**(r)
- List-based methods (position):
  - **first**()
  - **last**()
  - **before**(p), **after**(p)
  - **replaceElement**(p, o)
  - **swapElements**(p, q)
  - **insertBefore**(p, o)
  - **insertAfter**(p, o)
  - **insertFirst**(o)
  - **insertLast**(o)
  - **remove**(p)
- Bridge methods:
  - **atRank**(r) (*position of rank*)
  - **rankOf**(p) (*rank of position*)

Applications of Sequences

- The Sequence ADT is a basic, general-purpose, data structure for storing an ordered collection of elements
- Direct applications:
  - Generic replacement for stack, queue, vector, or list
  - small database (e.g., address book)
- Indirect applications:
  - Building blocks of more complex data structures

Array-based Implementation

- We use a circular array storing *positions*
- A position object stores:
  - Rank
  - Element
- Indices f and l keep track of first and last positions



Sequence Implementations

Operation	Array	List
size, isEmpty	1	1
atRank, rankOf, elemAtRank	1	***n***
first, last, before, after	1	1
replaceElement, swapElements	1	1
replaceAtRank	1	***n***
insertAtRank, removeAtRank	***n***	***n***
insertFirst, insertLast	1	1
insertAfter, insertBefore	***n***	1
remove	***n***	1

Space usage is more efficient for lists

Extra

Iterators

- An iterator abstracts the process of scanning through a collection of elements
- Methods of the ObjectIterator ADT:
  - object object()
  - boolean hasNext()
  - object nextObject()
  - reset()
- Extends the concept of Position by adding a traversal capability
- Implementation with an array or singly linked list
- An iterator is typically associated with another data structure
- We can augment the Stack, Queue, Vector, List and Sequence ADTs with method:
  - ObjectIterator elements()
- Two notions of iterator:
  - snapshot: freezes the contents of the data structure at a given time
  - dynamic: follows changes to the data structure